

Practical – Week10

1. Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
kavya_aluru@Pandu:~$ echo "Hello Linux Inodes" > file1.txt
kavya_aluru@Pandu:~$ ls -li file1.txt
40714 file1.txt
```

```
kavya_aluru@Pandu:~$ stat file1.txt
  File: file1.txt
  size: 19          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 40714         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/kavya_aluru)   Gid: ( 1000/kavya_aluru)
Access: 2026-09-27 07:51:25.541868214 +0000
Modify: 2026-09-27 07:51:25.561868212 +0000
Change: 2026-09-27 07:51:25.561868212 +0000
 Birth: 2026-09-27 07:51:25.541868214 +0000
```

```
kavya_aluru@Pandu:~$ ln file1.txt hardlink.txt
kavya_aluru@Pandu:~$ ls -li file1.txt hardlink.txt
40714 file1.txt 40714 hardlink.txt
kavya_aluru@Pandu:~$ stat file1.txt
  File: file1.txt
  size: 19          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 40714         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/kavya_aluru)   Gid: ( 1000/kavya_aluru)
Access: 2026-09-27 07:51:25.541868214 +0000
Modify: 2026-09-27 07:51:25.561868212 +0000
Change: 2026-09-27 07:51:41.665865805 +0000
 Birth: 2026-09-27 07:51:25.541868214 +0000
```

```
kavya_aluru@Pandu:~$ echo "Hard link test" >> hardlink.txt
kavya_aluru@Pandu:~$ cat file1.txt
Hello Linux Inodes
Hard link test
Hard link test
```

```

kavya_aluru@Pandu:~$ ln -s file1.txt symlink.txt
kavya_aluru@Pandu:~$ ls -i file1.txt hardlink.txt symlink.txt
40714 file1.txt 40714 hardlink.txt 40717 symlink.txt
kavya_aluru@Pandu:~$ stat symlink.txt
  File: 'symlink.txt' -> 'file1.txt'
  size: 9               Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48    Inode: 40717        Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/kavya_aluru)   Gid: ( 1000/kavya_aluru)
Access: 2026-09-27 07:55:43.301843127 +0000
Modify: 2026-09-27 07:55:37.949843802 +0000
Change: 2026-09-27 07:55:37.949843802 +0000
 Birth: 2026-09-27 07:55:37.949843802 +0000
kavya_aluru@Pandu:~$ stat -c '%i %h %F %n' symlink.txt
40717 1 symbolic link symlink.txt

```

```

kavya_aluru@Pandu:~$ find . -maxdepth 1 -printf '%i %p\n'
735 .
49016 ./git
12946 ./path_search
1043 ./process.c.save
13202 ./navigate.sh
49019 ./viminfo
40163 ./error.txt
2785 ./trace.txt
38686 ./combined_redirection
16129 ./command_executor.c
40717 ./symlink.txt
12984 ./ls_grep.c
44906 ./OS_Lab
45136 ./local
207 ./FileCopy
12948 ./variable_expand
12953 ./variable_expand.c
40089 ./MemoryMonitor
12907 ./producer_consumer.c
8906 ./process_state
2742 ./sample.txt
44900 ./profile
1029 ./filecopy.c
1006 ./process
44928 ./motd_shown
1078 ./process.c
2740 ./zombie.c
12954 ./builtin
44922 ./landscape
12671 ./waitpid_demo
2735 ./filecopy
44933 ./bash_history
51779 ./shell_project
12947 ./path_search.c
14135 ./demo.fifo
488 ./zombie

```

```

kavya_aluru@Pandu:~$ find . -inum "$(ls -i file1.txt | awk '{print $1}')"
./file1.txt
./hardlink.txt

```

```

kavya_aluru@Pandu:~$ rm hardlink.txt
kavya_aluru@Pandu:~$ stat file1.txt
  File: file1.txt
  size: 49          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 40714         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/kavya_aluru)   Gid: ( 1000/kavya_aluru)
Access: 2026-09-27 07:54:44.285849709 +0000
Modify: 2026-09-27 07:54:38.593850478 +0000
Change: 2026-09-27 07:59:45.577804748 +0000
 Birth: 2026-09-27 07:51:25.541868214 +0000
kavya_aluru@Pandu:~$ rm symlink.txt
kavya_aluru@Pandu:~$ ls -l file1.txt
-rw-r--r-- 1 kavya_aluru kavya_aluru 49 Sep 27 07:54 file1.txt

```

Report: The inode structure was investigated using `ls -i`, `stat`, and `find` commands. Hard links shared the same inode, while symbolic links had separate inodes. The experiment demonstrated how link creation and deletion affect inode allocation.

2. Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>

int main() {
    int src, dest;
    struct stat st;
    char *source, *output;
    src = open("input.txt", O_RDONLY);
    if (src < 0) {
        perror("Source file error");
        return 1;
    }
    fstat(src, &st);
    size_t size = st.st_size;
    if (size == 0) {
        printf("Empty file\n");
        close(src);
        return 0;
    }
    dest = open("output.txt",
                O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (dest < 0) {
        perror("Destination error");
        close(src);
        return 1;
    }
    ftruncate(dest, size);
    source = mmap(NULL, size, PROT_READ,
                  MAP_PRIVATE, src, 0);
    output = mmap(NULL, size, PROT_READ | PROT_WRITE,
                  MAP_SHARED, dest, 0);
    if (source == MAP_FAILED || output == MAP_FAILED) {
```

```
kavya_aluru@Pandu:~$ nano mmap_demo.c
kavya_aluru@Pandu:~$ echo "Hello Linux Memory Mapping" > input.txt
kavya_aluru@Pandu:~$ cat input.txt
Hello Linux Memory Mapping
kavya_aluru@Pandu:~$ gcc mmap_demo.c -o mmap_demo
kavya_aluru@Pandu:~$ ./mmap_demo
File copied successfully using mmap()
File size: 27 bytes
Copy time: 0.000218 seconds
```

```
GNU nano 8.7.1 traditional.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

int main() {
    int src, dest;
    char buffer[4096];
    ssize_t n;

    src = open("input.txt", O_RDONLY);
    dest = open("output2.txt",
               O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (src < 0 || dest < 0) {
        perror("File error");
        return 1;
    }

    clock_t start = clock();

    while ((n = read(src, buffer, sizeof(buffer))) > 0) {
        write(dest, buffer, n);
    }

    clock_t end = clock();

    printf("File copied using read() and write()\n");
    printf("Copy time: %.6f seconds\n",
           (double)(end - start) / CLOCKS_PER_SEC);

    close(src);
    close(dest);

    return 0;
}
```

```
kavya_aluru@Pandu:~$ nano traditional.c
kavya_aluru@Pandu:~$ gcc traditional.c -o traditional
kavya_aluru@Pandu:~$ ./traditional
File copied using read() and write()
Copy time: 0.000025 seconds
```

```
kavya_aluru@Pandu:~$ time ./mmap_demo
File copied successfully using mmap()
File size: 27 bytes
Copy time: 0.000517 seconds

real    0m0.009s
user    0m0.003s
sys     0m0.000s
kavya_aluru@Pandu:~$ time ./traditional
File copied using read() and write()
Copy time: 0.000022 seconds

real    0m0.002s
user    0m0.001s
sys     0m0.001s
```

Report: The file was successfully read and written using mmap() and traditional read() and write() system calls. Both methods were verified using cmp, and their execution times were compared using time.